Week 3 - In Class Activity: Spatial Autocorrelation with the Meuse Dataset

Goal

In this activity, you will open, explore, model, and diagnose spatial autocorrelation in a point dataset using R.

We will use the Meuse dataset, which contains point observations of topsoil heavy-metal concentrations from a floodplain along the Meuse River near Stein, Netherlands. The data include spatial coordinates, heavy-metal concentrations such as zinc, cadmium, copper, and lead, plus environmental covariates such as distance to river, soil type, and flooding frequency.

The central question is:

After fitting a simple non-spatial regression model, do the residuals still show spatial patterning?

If they do, then the model has not fully explained the spatial structure in zinc concentrations.

1. Setup

This activity uses `tidymodels`, the most common modern machine-learning framework in R. `tidymodels` provides a consistent workflow for preprocessing, modeling, fitting, prediction, and model evaluation.

We also use several common spatial packages:

- `sf` stores and maps spatial data.
- `spdep` creates spatial neighbor weights and calculates Moran's I.
- `gstat` calculates the variogram.
- `ggplot2`, included with `tidymodels`, creates plots.

```
# Install packages if needed:
# install.packages(c("tidymodels", "sf", "spdep", "gstat"))
#install.packages(c("spdep", "gstat"))
library(tidymodels)
library(sf)
library(spdep)
library(gstat)

theme_set(theme_minimal())
```

2. Load the Meuse Data

The original `x` and `y` coordinates are in **EPSG:28992**, also called the Dutch national grid / RD New coordinate system.

```
meuse <- read.csv(
  "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/meuse.csv"
)

meuse_sf <- st_as_sf(
  meuse,
  coords = c("x", "y"),
```

```
  crs = 28992,
  remove = FALSE
)

meuse_sf
```

```
## Simple feature collection with 155 features and 14 fields
## Geometry type: POINT
## Dimension:     XY
## Bounding box:  xmin: 178605 ymin: 329714 xmax: 181390 ymax: 333611
## Projected CRS: Amersfoort / RD New
## First 10 features:
##         x      y cadmium copper lead zinc  elev       dist   om ffreq soil lime
## 1  181072 333611    11.7     85  299 1022 7.909 0.00135803 13.6     1    1    1
## 2  181025 333558     8.6     81  277 1141 6.983 0.01222430 14.0     1    1    1
## 3  181165 333537     6.5     68  199  640 7.800 0.10302900 13.0     1    1    1
## 4  181298 333484     2.6     81  116  257 7.655 0.19009400  8.0     1    2    0
## 5  181307 333330     2.8     48  117  269 7.480 0.27709000  8.7     1    2    0
## 6  181390 333260     3.0     61  137  281 7.791 0.36406700  7.8     1    2    0
## 7  181165 333370     3.2     31  132  346 8.217 0.19009400  9.2     1    2    0
## 8  181027 333363     2.8     29  150  406 8.490 0.09215160  9.5     1    1    0
## 9  181060 333231     2.4     37  133  347 8.668 0.18461400 10.6     1    1    0
## 10 181232 333168     1.6     24   80  183 9.049 0.30970200  6.3     1    2    0
##    landuse dist.m              geometry
## 1       Ah     50 POINT (181072 333611)
## 2       Ah     30 POINT (181025 333558)
## 3       Ah    150 POINT (181165 333537)
## 4       Ga    270 POINT (181298 333484)
## 5       Ah    380 POINT (181307 333330)
## 6       Ga    470 POINT (181390 333260)
## 7       Ah    240 POINT (181165 333370)
## 8       Ab    120 POINT (181027 333363)
## 9       Ab    240 POINT (181060 333231)
## 10       W    420 POINT (181232 333168)
```

What to Notice

After loading the data, we should have one row per soil sample. The `geometry` column stores the point location for each sample. The variables `x` and `y` are coordinates, while variables such as `zinc`, `cadmium`, `copper`, and `lead` are measured concentrations at those locations.

Also notice which variables are categorical and which are continuous. Determine which columns would need to be converted to factors or dummy variables before modeling.

Important variables:

- `elev`: relative elevation above the local river bed in meters.
- `dist`: normalized distance to the Meuse River, scaled between 0 and 1.
- `dist.m`: actual distance to the Meuse River in meters.
- `om`: organic matter content as a percentage.
- `ffreq`: flooding frequency class.
- `soil`: soil type. This should not be treated as a continuous numeric variable.
- `lime`: lime class, where 0 = absent and 1 = present.
- `landuse`: land use class.

3. Data Exploration

Dataset Structure

This summary tells us how many observations and variables are present, what the column names are, which variables are numeric or categorical, and whether there are missing values.

```
glimpse(meuse_sf)
```

```
## Rows: 155
## Columns: 15
## $ x        <int> 181072, 181025, 181165, 181298, 181307, 181390, 181165, 181027, 1810~
## $ y        <int> 333611, 333558, 333537, 333484, 333330, 333260, 333370, 333363, 3332~
## $ cadmium  <dbl> 11.7, 8.6, 6.5, 2.6, 2.8, 3.0, 3.2, 2.8, 2.4, 1.6, 1.4, 1.8, 11.2, 2~
## $ copper   <int> 85, 81, 68, 81, 48, 61, 31, 29, 37, 24, 25, 25, 93, 31, 27, 86, 74, ~
## $ lead     <int> 299, 277, 199, 116, 117, 137, 132, 150, 133, 80, 86, 97, 285, 183, 1~
## $ zinc     <int> 1022, 1141, 640, 257, 269, 281, 346, 406, 347, 183, 189, 251, 1096, ~
## $ elev     <dbl> 7.909, 6.983, 7.800, 7.655, 7.480, 7.791, 8.217, 8.490, 8.668, 9.049~
## $ dist     <dbl> 0.00135803, 0.01222430, 0.10302900, 0.19009400, 0.27709000, 0.364067~
## $ om       <dbl> 13.6, 14.0, 13.0, 8.0, 8.7, 7.8, 9.2, 9.5, 10.6, 6.3, 6.4, 9.0, 15.4~
## $ ffreq    <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ soil     <int> 1, 1, 1, 2, 2, 2, 2, 1, 1, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ lime     <int> 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ landuse  <chr> "Ah", "Ah", "Ah", "Ga", "Ah", "Ga", "Ah", "Ab", "Ab", "W", "Fh", "Ag~
## $ dist.m   <int> 50, 30, 150, 270, 380, 470, 240, 120, 240, 420, 400, 300, 20, 130, 2~
## $ geometry <POINT [m]> POINT (181072 333611), POINT (181025 333558), POINT (181165 33~
```

```
cat("\nShape:\n")
```

```
##
## Shape:
```

```
dim(meuse_sf)
```

```
## [1] 155  15
```

```
cat("\nMissing values per column:\n")
```

```
##
## Missing values per column:
```

```
colSums(is.na(meuse_sf)) %>% sort(decreasing = TRUE)
```

```
##       om  landuse        x        y  cadmium   copper     lead     zinc     elev
##        2        1        0        0        0        0        0        0        0
##     dist    ffreq     soil     lime   dist.m geometry
##        0        0        0        0        0        0
```

```
#om has 2 missing values
```

Numeric Summaries

Environmental concentration data are often right-skewed: a few locations may have much higher concentrations than most locations. That is one reason we later model `log_zinc` instead of raw `zinc`.

```
# TODO: get numeric summaries for each numeric column.
# Include count, mean, median, standard deviation, min, max, and useful quantiles.
summary(meuse_sf)
```

```
##        x                y             cadmium           copper            lead
```

```
##  Min.   :178605   Min.   :329714   Min.   : 0.200   Min.   : 14.00   Min.   : 37.0
##  1st Qu.:179371   1st Qu.:330762   1st Qu.: 0.800   1st Qu.: 23.00   1st Qu.: 72.5
##  Median :179991   Median :331633   Median : 2.100   Median : 31.00   Median :123.0
##  Mean   :180005   Mean   :331635   Mean   : 3.246   Mean   : 40.32   Mean   :153.4
##  3rd Qu.:180630   3rd Qu.:332463   3rd Qu.: 3.850   3rd Qu.: 49.50   3rd Qu.:207.0
##  Max.   :181390   Max.   :333611   Max.   :18.100   Max.   :128.00   Max.   :654.0
##
##       zinc             elev             dist              om             ffreq
##  Min.   : 113.0   Min.   : 5.180   Min.   :0.00000   Min.   : 1.000   Min.   :1.000
##  1st Qu.: 198.0   1st Qu.: 7.546   1st Qu.:0.07569   1st Qu.: 5.300   1st Qu.:1.000
##  Median : 326.0   Median : 8.180   Median :0.21184   Median : 6.900   Median :1.000
##  Mean   : 469.7   Mean   : 8.165   Mean   :0.24002   Mean   : 7.478   Mean   :1.606
##  3rd Qu.: 674.5   3rd Qu.: 8.955   3rd Qu.:0.36407   3rd Qu.: 9.000   3rd Qu.:2.000
##  Max.   :1839.0   Max.   :10.520   Max.   :0.88039   Max.   :17.000   Max.   :3.000
##                                                      NA's   :2
##       soil             lime          landuse             dist.m
##  Min.   :1.000   Min.   :0.0000   Length:155         Min.   :  10.0
##  1st Qu.:1.000   1st Qu.:0.0000   Class :character   1st Qu.:  80.0
##  Median :1.000   Median :0.0000   Mode  :character   Median : 270.0
##  Mean   :1.452   Mean   :0.2839                      Mean   : 290.3
##  3rd Qu.:2.000   3rd Qu.:1.0000                      3rd Qu.: 450.0
##  Max.   :3.000   Max.   :1.0000                      Max.   :1000.0
##
##           geometry
##  POINT        :155
##  epsg:28992   :  0
##  +proj=ster...:  0
##
##
##
##
```

```
# missing count, SD

summary_values <- meuse_sf %>%
  st_drop_geometry() %>%
  select(-x, -y) %>%
  select_if(is.numeric) %>%
  summarize(across(
    everything(),
          list(
        count = ~sum(!is.na(.)),
        mean  = ~mean(., na.rm = TRUE),
        sd    = ~sd(., na.rm = TRUE),
        max   = ~max(., na.rm = TRUE),
        min   = ~min(., na.rm = TRUE),
        q_25 = ~quantile(., na.rm = TRUE)[[2]],
        q_75 = ~quantile(., na.rm = TRUE)[[4]]
      ),
      .names = "{.col}_{.fn}"
  )
  )
summary_values <- pivot_longer(summary_values,cols = colnames(summary_values),
    names_to = c("column")
```

```
    )
#summary_values$column
#glimpse(summary_values)
```

Pairwise Relationships

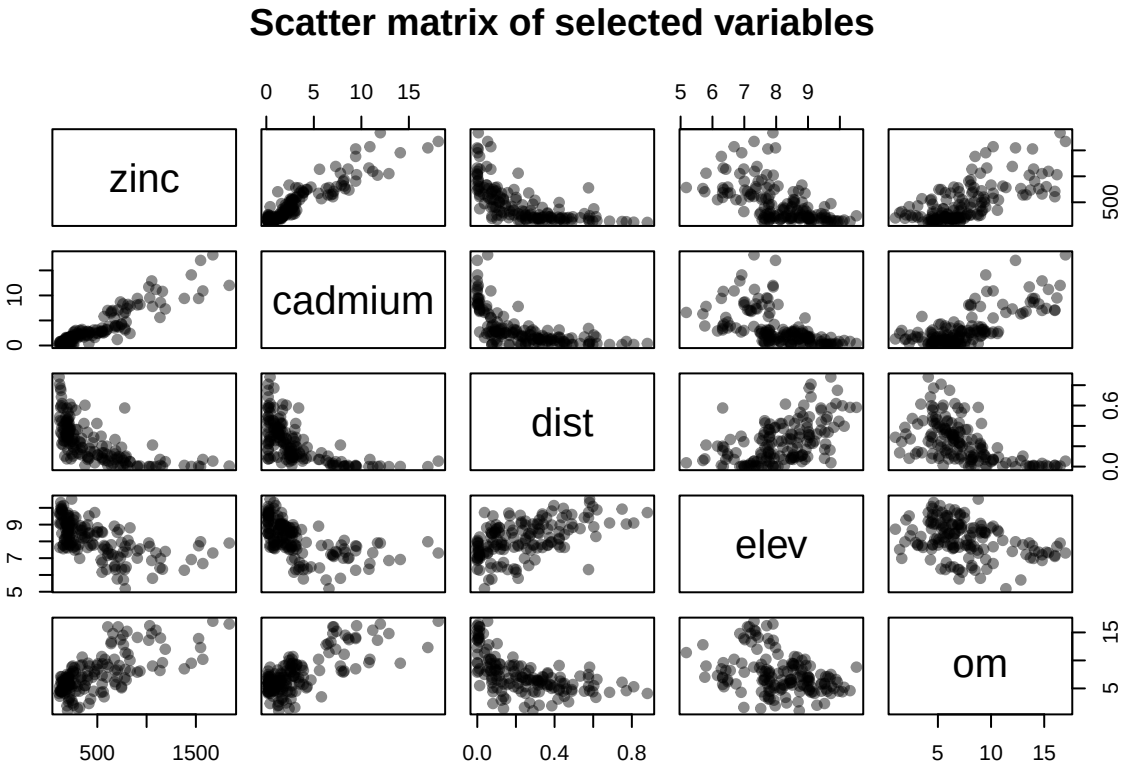
The scatter matrix lets us compare zinc to distance from the river and to other metal concentrations.

Typical patterns to look for:

- Strong positive relationships among heavy metals may suggest a shared contamination process.
- A relationship between `dist` and `zinc` may suggest that contamination is related to distance from the river.
- Curved or fan-shaped relationships may suggest that a transformation, such as a logarithm, is useful.

```
pair_vars <- c("zinc", "cadmium", "dist", "elev", "om")

meuse %>%
  select(all_of(pair_vars)) %>%
  drop_na() %>%
  pairs(
    main = "Scatter matrix of selected variables",
    pch = 19,
    col = rgb(0, 0, 0, 0.45)
  )
```



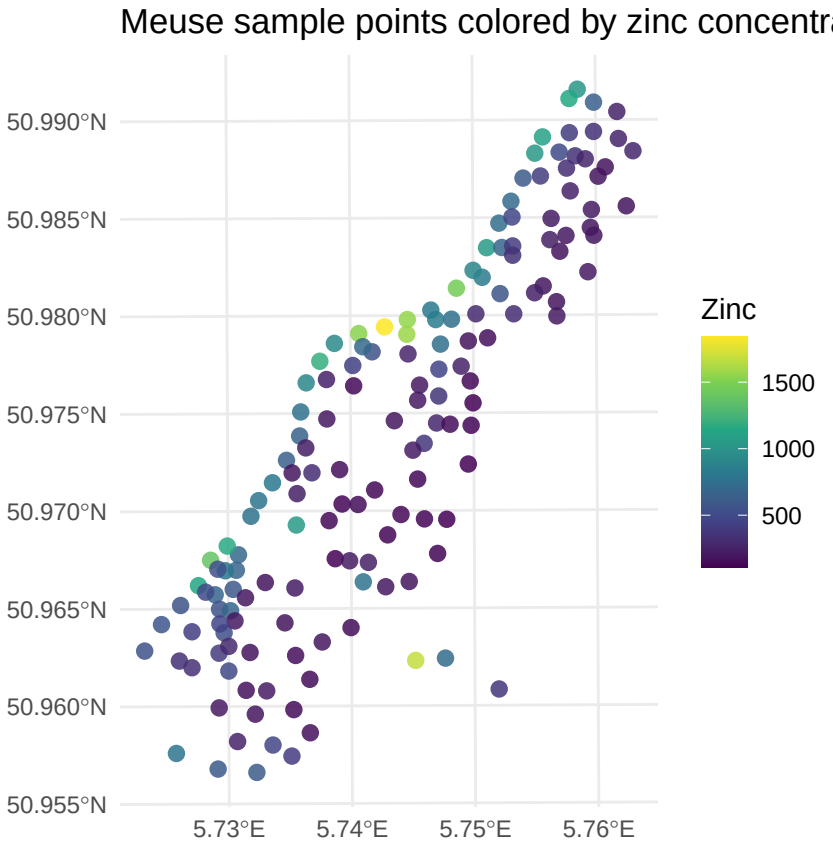
Spatial View

This map places the sample points in their geographic positions and colors them by zinc concentration.

Interpretation guide:

- Points with similar colors near one another suggest spatial clustering.
- Higher zinc values concentrated in one part of the floodplain suggest that location matters.
- If high values occur near the river or flood-prone areas, that supports the idea that river transport and deposition may influence contamination.

```
ggplot(meuse_sf) +
  geom_sf(aes(color = zinc), size = 2.4, alpha = 0.85) +
  scale_color_viridis_c() +
  labs(
    title = "Meuse sample points colored by zinc concentration",
    color = "Zinc"
  )
```



4. Model Preprocessing and Regression

We now build a simple non-spatial model for zinc concentration.

The response variable is `log_zinc`, the natural logarithm of zinc. This transformation is common for concentration data because it reduces right-skewness and makes large values less dominant in the regression.

The predictors are:

- `dist`: distance to the river, treated as numeric.
- `soil`: soil class, treated as categorical.
- `ffreq`: flooding frequency class, treated here as ordinal numeric.
- `elev`: relative elevation.
- `om`: organic matter content.

This model is intentionally simple. It asks:

How much of the zinc pattern can be explained by basic environmental covariates before explicitly modeling spatial dependence?

```
df <- meuse_sf %>%
  st_drop_geometry() %>%
  mutate(
    log_zinc = log(zinc),
    soil = factor(soil),
    ffreq = as.numeric(ffreq)
  )

feature_cols <- c("dist", "soil", "ffreq", "elev", "om")

df %>%
  select(log_zinc, all_of(feature_cols)) %>%
  glimpse()
```

```
## Rows: 155
## Columns: 6
## $ log_zinc <dbl> 6.929517, 7.039660, 6.461468, 5.549076, 5.594711, 5.638355, 5.846439~
## $ dist     <dbl> 0.00135803, 0.01222430, 0.10302900, 0.19009400, 0.27709000, 0.364067~
## $ soil     <fct> 1, 1, 1, 2, 2, 2, 2, 1, 1, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ ffreq    <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ elev     <dbl> 7.909, 6.983, 7.800, 7.655, 7.480, 7.791, 8.217, 8.490, 8.668, 9.049~
## $ om       <dbl> 13.6, 14.0, 13.0, 8.0, 8.7, 7.8, 9.2, 9.5, 10.6, 6.3, 6.4, 9.0, 15.4~
```

Build the `tidymodels` Recipe and Workflow

The recipe separates preprocessing from model fitting.

For numeric predictors, the workflow imputes missing values with the median and standardizes the values. Standardization is not required for ordinary linear regression, but it is useful practice in many machine-learning workflows.

For categorical predictors, the workflow fills missing values with the most common category and then creates dummy variables.

```
zinc_recipe <- recipe(log_zinc ~ dist + soil + ffreq + elev + om, data = df) %>%
  step_impute_median(all_numeric_predictors()) %>%
  step_impute_mode(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())

lm_spec <- linear_reg() %>%
  set_engine("lm")

zinc_workflow <- workflow() %>%
  add_recipe(zinc_recipe) %>%
  add_model(lm_spec)

zinc_workflow
```

```
## == Workflow ====================================================================
## Preprocessor: Recipe
## Model: linear_reg()
##
## -- Preprocessor ----------------------------------------------------------------
```

```
## 4 Recipe Steps
##
## * step_impute_median()
## * step_impute_mode()
## * step_dummy()
## * step_normalize()
##
## -- Model ---------------------------------------------------------------------
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

Fit the Model and Calculate Residuals

After fitting the model, we calculate two quantities:

- `pred_m1`: the model's predicted log zinc value.
- `resid_m1`: the residual, calculated as observed log zinc minus predicted log zinc.

Residuals are central to the spatial diagnostic step. If the model fully explains the systematic pattern in zinc, the residuals should look roughly random in space. If residuals are still spatially clustered, the model is missing spatial structure.

```
zinc_fit <- fit(zinc_workflow, data = df)

df <- df %>%
  bind_cols(predict(zinc_fit, new_data = df)) %>%
  rename(pred_m1 = .pred) %>%
  mutate(resid_m1 = log_zinc - pred_m1)

regression_metrics <- metric_set(rsq_trad, rmse)

regression_metrics(
  df,
  truth = log_zinc,
  estimate = pred_m1
)
```

```
## # A tibble: 2 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 rsq_trad standard       0.741
## 2 rmse     standard       0.366
```

Interpreting the Model Fit

`R^2` measures the proportion of variation in `log_zinc` explained by the predictors. Larger values mean the model explains more variation, but `R^2` alone does not tell us whether the model assumptions are satisfied.

`RMSE` measures the typical prediction error on the log-zinc scale. Smaller values indicate better fit.

Important caution: these metrics are calculated after fitting and predicting on the same dataset. They describe in-sample fit, not out-of-sample predictive performance. That is why we also use cross-validation below.

Cross-Validation

Cross-validation gives a more realistic estimate of predictive performance. The dataset is split into five parts. The model is trained on four parts and tested on the remaining part, repeated five times.

The mean cross-validated `R^2` is usually lower than the in-sample `R^2`. If it is much lower, the model may not generalize well.

```
set.seed(42)

folds <- vfold_cv(df, v = 5)

cv_results <- fit_resamples(
  zinc_workflow,
  resamples = folds,
  metrics = regression_metrics
)

collect_metrics(cv_results)
```

```
## # A tibble: 2 x 6
##   .metric  .estimator  mean     n std_err .config
##   <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1 rmse     standard   0.383     5  0.0141 pre0_mod0_post0
## 2 rsq_trad standard   0.706     5  0.0173 pre0_mod0_post0
```

5. Spatial Diagnostics: Neighborhood Weights

To measure spatial autocorrelation, we first define which observations are neighbors.

Here, each sample point is connected to its six nearest neighboring points using k-nearest neighbors. The weights are row-standardized, meaning each point's neighbor weights sum to one.

This step matters because Moran's I depends on the neighborhood definition. A different number of neighbors could produce slightly different results.

```
library(spdep)
coords <- df %>%
  select(x, y) %>%
  as.matrix()

knn <- knearneigh(coords, k = 6)
neighbors <- knn2nb(knn)
weights <- nb2listw(neighbors, style = "W")
```

6. Moran's I

Moran's I measures whether similar values occur near one another.

Interpretation guide:

- Positive Moran's I: nearby observations tend to have similar values, indicating clustering.
- Moran's I near zero: little or no spatial autocorrelation.
- Negative Moran's I: nearby observations tend to be dissimilar, indicating spatial dispersion or checkerboard-like patterning.
- Small p-value, often below 0.05: the observed spatial pattern is unlikely under random spatial arrangement.

We calculate Moran's I twice:

1. For `log_zinc`, to test whether zinc itself is spatially clustered.
2. For `resid_m1`, to test whether the regression residuals are still spatially clustered after accounting for distance, soil, flooding frequency, elevation, and organic matter.

```
set.seed(42)

moran_raw <- moran.mc(df$log_zinc, weights, nsim = 999)
moran_resid <- moran.mc(df$resid_m1, weights, nsim = 999)

moran_raw
```

```
##
##  Monte-Carlo simulation of Moran I
##
## data:  df$log_zinc
## weights: weights
## number of simulations + 1: 1000
##
## statistic = 0.51996, observed rank = 1000, p-value = 0.001
## alternative hypothesis: greater
```

```
moran_resid # more important if positive and significant, that means model is not good and needs a spat
```

```
##
##  Monte-Carlo simulation of Moran I
##
## data:  df$resid_m1
## weights: weights
## number of simulations + 1: 1000
##
## statistic = 0.33039, observed rank = 1000, p-value = 0.001
## alternative hypothesis: greater
```

Interpreting the Moran's I Results

If `log_zinc` has a positive and statistically significant Moran's I, zinc concentrations are spatially clustered. That means nearby samples tend to have similar zinc levels.

The residual Moran's I is the more important diagnostic for the regression model:

- If residual Moran's I is not significant, then the covariates may have explained most of the spatial structure relevant to zinc.
- If residual Moran's I is still positive and significant, then the model has missed spatial structure. This suggests that nearby residuals are similar and that a spatial modeling approach may be needed.

In practical terms, significant residual spatial autocorrelation means the simple linear model may overstate how much independent information is present in the data. It can also produce overly optimistic uncertainty estimates because nearby observations are not fully independent.

7. Variogram of Residuals

A variogram describes how dissimilarity changes with distance. Here, we calculate a variogram for the model residuals.

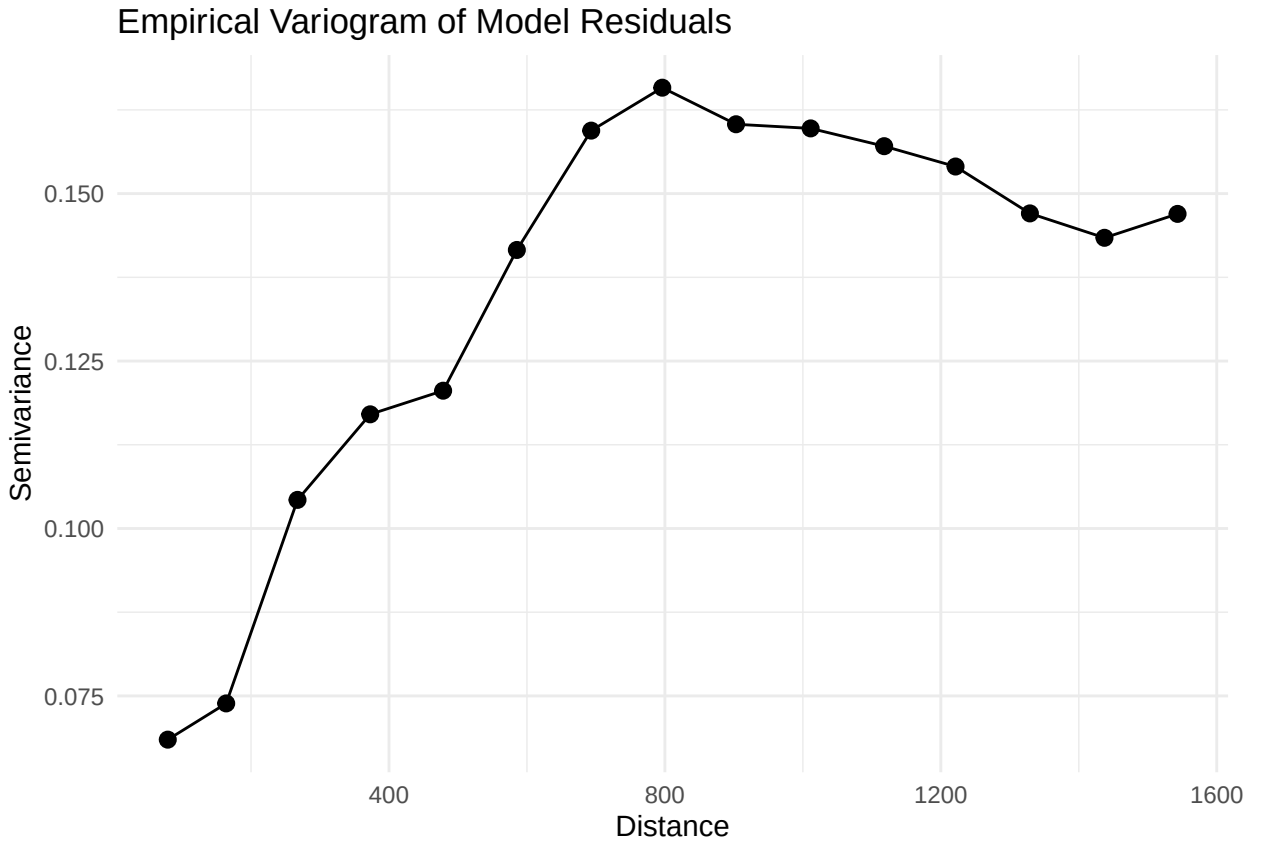
The y-axis shows semivariance, which is a measure of how different residuals are from one another. The x-axis shows distance between sample pairs.

Interpretation guide:

- If semivariance increases with distance, nearby residuals are more similar than distant residuals. This indicates spatial dependence.
- If the curve levels off, the distance where it levels off is related to the spatial range of dependence.
- A flat variogram suggests little spatial structure in the residuals.
- A high semivariance at very short distances may suggest measurement error or microscale variation.

```
vgm_resid <- variogram(
  resid_m1 ~ 1,
  locations = ~ x + y,
  data = df
)

ggplot(vgm_resid, aes(x = dist, y = gamma)) +
  geom_point(size = 2.5) +
  geom_line() +
  labs(
    title = "Empirical Variogram of Model Residuals",
    x = "Distance",
    y = "Semivariance"
  )
```



Interpreting the Variogram Results

The variogram complements Moran's I. Moran's I gives a single summary of spatial autocorrelation for a chosen neighborhood structure, while the variogram shows how spatial dependence changes over distance.

If the residual variogram rises with distance before flattening out, the residuals still contain spatial structure. That means the simple non-spatial regression has not captured all of the geographic process affecting zinc.

If the residual variogram is nearly flat, the residuals are not strongly distance-dependent. That would support the idea that the predictors explain most of the spatial pattern.

Together, the Moran's I and variogram results help answer the main question of the activity:

Does a simple non-spatial model remove the spatial structure from zinc concentrations, or is a spatial model still needed?

8. Overall Discussion

This workflow shows a common sequence in spatial data analysis:

1. Explore the data. Understand the variables, distributions, missing values, and spatial layout.
2. Fit a baseline non-spatial model. Use environmental covariates such as distance to river, soil type, and flooding frequency to explain zinc concentration.
3. Examine residuals. Residuals show what the model did not explain.
4. Test for spatial autocorrelation. Moran's I asks whether similar residuals occur near one another.
5. Examine spatial dependence by distance. The variogram shows whether residual similarity changes as locations get farther apart.

If both Moran's I and the variogram suggest residual spatial structure, then a more advanced model may be appropriate. Possible next steps include adding more predictors, using spatial lag or spatial error models, fitting a geostatistical model, or using kriging to model spatially structured residual variation.

The most important lesson is that strong model performance metrics do not automatically mean the model is spatially appropriate. Spatial diagnostics are needed because geographic observations are often not independent.

9. Reflection Questions

1. Why did we model `log_zinc` instead of raw `zinc`? The raw zinc values have a curved shape relationship to distance, where there are very high values at smaller distances and low values further away.
2. What does a positive Moran's I tell us about the spatial arrangement of values? A positive value indicates spatial clustering.
3. Why do we check Moran's I on the residuals, not only on the original zinc values? Checking Moran's I on residual values can determine if th emodel is attributing relationships that capture the spatial relationship with the environmental values.
4. If residual spatial autocorrelation remains, what does that imply about the regression model? It implies the model has not accurately captured the spatial structure of the data.
5. How does the variogram add information beyond Moran's I? It shows how at what distance the residuals are no longer correlated, rather than a binary answer of are the results still spatially correlated.
6. What might your next step be? Looking at the map and the scatter matrix, I would probably try to group the points with a few neighbors or by elevation when splitting the data to better understand the spatial relationship. If I knew more about zinc I'd suggest incorporating other features like flood frequency and soil type, although those are probably related to distance and elevation.